

TEMA 6. Genericidad

1. Empleando `void*` en C o `Object` en Java, pon un ejemplo de una estructura de datos, que empleando un array primitivo, permita alojar cualquier tipo de dato.

Respuesta

En C, se puede usar `void*` para almacenar cualquier tipo de dato en una estructura, ya que representa un puntero genérico. Sin embargo, el compilador pierde información sobre el tipo real, por lo que el programador debe hacer conversiones manuales.

En Java ocurre algo similar usando `Object`, ya que todas las clases heredan de él. Se puede crear una estructura que almacene referencias a `Object` y, por tanto, cualquier tipo.

```
class Contenedor {
    private Object[] datos;
    private int size;

    public Contenedor(int capacidad) {
        datos = new Object[capacidad];
        size = 0;
    }

    public void add(Object o) {
        datos[size++] = o;
    }

    public Object get(int i) {
        return datos[i];
    }
}
```

El problema es que al recuperar los elementos, se pierde el tipo concreto y es necesario hacer casting.

2. Brevemente, ¿Qué significa la **programación genérica**? ¿Es el ejemplo anterior un ejemplo básico de programación genérica?

Respuesta

La programación genérica consiste en escribir código que funcione con distintos tipos de datos sin necesidad de duplicarlo. En lugar de fijar un tipo concreto, se parametriza el tipo.

El ejemplo anterior es una forma básica de genericidad, pero no es segura. Aunque permite trabajar con distintos tipos, no ofrece control de tipos en tiempo de compilación.

Por tanto, es una aproximación, pero no es la solución moderna ni recomendable

frente al uso de generics.

3. Indica los problemas respecto al chequeo de tipos, de emplear ``void*`` o ``Object`` cuando se crean estructuras de datos genéricas.

Respuesta

El principal problema es la pérdida de información de tipos. El compilador no puede verificar si se está usando correctamente un objeto, lo que puede provocar errores en tiempo de ejecución.

Esto obliga a hacer downcasting, lo que introduce riesgo de errores si el tipo real no coincide con el esperado. Además, el código se vuelve menos legible y más propenso a fallos.

En C, esto es aún más peligroso, ya que no existe comprobación en tiempo de ejecución, lo que puede provocar errores graves de memoria.

4. Vamos entonces con mecanismos de mejora de la programación genérica ¿Qué son los `**parámetros de tipo**`?

Respuesta

Los parámetros de tipo son una forma de indicar que una clase o método trabaja con un tipo genérico que se especificará más adelante.

Se representan normalmente con letras como `<T>`, `<E>`, etc. Esto permite al compilador conocer el tipo concreto en cada uso.

Gracias a esto, se obtiene seguridad de tipos en tiempo de compilación, evitando errores de casting.

5. En Java existe "generics", en C++ existen "templates". Pon un ejemplo de uso de programación genérica en ambos, instanciando una lista o vector dinámico que solo admite ``String``. Introduce valores, y luego haz un recorrido de ellos mostrando cómo cada elemento es del tipo concreto con seguridad.

Respuesta

En Java, se puede usar una lista genérica:

```
import java.util.*;

List<String> lista = new ArrayList<>();
lista.add("Hola");
lista.add("Mundo");

for (String s : lista) {
    System.out.println(s.toUpperCase());
}
```

En C++, se usan templates:

```
#include <vector>
#include <string>
#include <iostream>

std::vector<std::string> lista;
lista.push_back("Hola");
lista.push_back("Mundo");

for (const std::string& s : lista) {
    std::cout << s << std::endl;
}
```

En ambos casos, el tipo está garantizado y no se necesita casting.

6. Sobre el funcionamiento de la programación genérica. ¿Qué hace el compilador cuando se instancia una clase que tiene parámetros de tipo? ¿Hace lo mismo C++ y Java? ¿Qué es el "type erasure" de Java y la "instanciación de plantillas" de C++?

Respuesta

En C++, los templates se instancian generando código específico para cada tipo. Es decir, el compilador crea versiones concretas del código.

En Java, en cambio, se usa type erasure. Esto significa que los tipos genéricos se eliminan en compilación y se reemplazan por Object o el tipo límite.

Esto implica que en tiempo de ejecución no existe información sobre el tipo genérico.

7. Vamos a crear una nueva clase con parámetros de tipo. Define en Java una clase `Par`, que permite alojar dos valores de tipos diferentes. Incluye un constructor y un getter para cada tipo. Pon un ejemplo de uso de ese `Par`, por ejemplo para especificar el tipo de retorno de una función que devuelve en un `Par` la media y desviación típica de un array de `double`.

Respuesta

Se puede definir una clase genérica con dos tipos distintos:

```
class Par<A, B> {
    private final A primero;
    private final B segundo;

    public Par(A primero, B segundo) {
        this.primero = primero;
        this.segundo = segundo;
    }

    public A getPrimero() { return primero; }
    public B getSegundo() { return segundo; }
}
```

Ejemplo de uso:

```
public static Par<Double, Double> calcular(double[] datos) {
    double media = 0;
    for (double d : datos) media += d;
```

```

        media /= datos.length;

        return new Par<>(media, 0.0);
    }

```

Esto permite devolver dos valores con tipos distintos de forma segura.

8. En Java, se pueden declarar parámetros de tipo también a nivel de método, no solo a nivel de clase. Pon un ejemplo con un método genérico `seleccionaUno`, que pasados dos objetos del mismo tipo, te devuelva aleatoriamente uno de ellos. Muestra la diferencia de definirlo con dos `Object`, a definirlo con dos parámetros de tipo, en terminos de (i) evitar downcasting y (ii) forzar que ambos objetos sean del mismo tipo.

Respuesta
Sin generics:

```

public static Object seleccionaUno(Object a, Object b) {
    return Math.random() < 0.5 ? a : b;
}

```

Con generics:

```

public static <T> T seleccionaUno(T a, T b) {
    return Math.random() < 0.5 ? a : b;
}

```

La versión genérica evita casting y obliga a que ambos parámetros sean del mismo tipo.

9. ¿Se pueden establecer restricciones en los parámetros de tipo? Por ejemplo, si quiero definir un tipo genérico `<T>`, ¿puedo decir que tenga que ser, al menos, un número para poder tratarlo como tal? Pon un ejemplo en Java de un `Punto` con dos coordenadas, metodos `getX`, `getY`, y una función `calcularDistanciaA` otro `Punto`. Permite que esas coordenadas sean cualquier tipo de número. Pon dos soluciones: una simplemente creando coordenadas de tipo `Number` y otra añadiendo generics para reforzar el chequeo de tipos y saber exactamente con qué tipo de número trabaja el `Punto`. En este caso y respecto al "type erasure", ¿cuál es el tipo final tras la compilación?

Respuesta
Sí, se pueden restringir con extends. Por ejemplo:

```

class Punto<T extends Number> {
    private final T x, y;

    public Punto(T x, T y) {
        this.x = x;
        this.y = y;
    }

    public double distancia(Punto<T> p) {
        double dx = x.doubleValue() - p.x.doubleValue();

```

```

        double dy = y.doubleValue() - p.y.doubleValue();
        return Math.sqrt(dx*dx + dy*dy);
    }
}

```

Tras compilación, debido a type erasure, el tipo pasa a ser Number.

10. Sobre las soluciones anteriores. Si bien ambas permiten trabajar con distintos tipos de número sin duplicar la clase `Punto`, reflexiona sobre el refuerzo del chequeo de tipos con generics. ¿Permiten ambas crear un punto con una coordenada de tipo entero y la otra coordenada de tipo real? ¿Qué tipo devuelve el `getX` con la solución sin generics y qué tipo devuelve el que tiene la solución con generics?

Respuesta

Sin generics (Number), se podrían mezclar tipos como Integer y Double. Esto reduce la seguridad.

Con generics (<T extends Number>), se obliga a que ambas coordenadas sean del mismo tipo.

Además, getX() devuelve Number en el primer caso, pero T en el segundo, lo que mejora la precisión de tipos.

11. Hagamos un ejemplo avanzado. El siguiente código, con interfaz `Punto`, que define un método `calcularDistanciaA(Punto p)`, junto con las implementaciones `Punto2D` y `Punto3D`. Añade generics para asegurarnos que la sobreescritura del método calcular distancia a otro `Punto` siempre es sobre un `Punto` del mismo tipo, evitando `instanceof` y el downcasting.

```

```java
public interface Punto {
 public double distanciaA(Punto p);
}

public class Punto2D implements Punto {
 private final double x, y;
 public Punto2D(double x, double y) {
 this.x = x; this.y = y;
 }

 @Override
 public double distanciaA(Punto p) {
 if (p instanceof Punto2D) {
 Punto2D p2d = (Punto2D) p;
 return Math.sqrt(Math.pow(x - p2d.x, 2)
 + Math.pow(y - p2d.y, 2));
 } else {
 throw new RuntimeException("p debe ser Punto 2D");
 }
 }
}

public class Punto3D implements Punto {

```

```
// Igual que Punto2D, pero con tres coordenadas
...
}
```

## Respuesta

Se puede usar generics para evitar casting:

```
interface Punto<T extends Punto<T>> {
 double distanciaA(T p);
}

class Punto2D implements Punto<Punto2D> {
 private final double x, y;

 public Punto2D(double x, double y) {
 this.x = x;
 this.y = y;
 }

 public double distanciaA(Punto2D p) {
 double dx = x - p.x;
 double dy = y - p.y;
 return Math.sqrt(dx*dx + dy*dy);
 }
}
```

Esto garantiza que solo se comparen puntos del mismo tipo.

12. Dado que **String** es subtipo de **Object**, ¿significa eso que **List<String>** es subtipo de **List<Object>**? ¿Y que **String[]** es subtipo de **Object[]**? Razona por qué la respuesta es diferente en cada caso y qué problema en tiempo de ejecución puede aparecer con los arrays. A partir de estos ejemplos, define qué significa que un tipo genérico sea **covariante**, **contravariante** o **invariante** respecto a su parámetro de tipo.

## Respuesta

List no es subtipo de List, pero String[] sí es subtipo de Object[].

Esto se debe a que los arrays son covariantes, lo que puede causar errores en tiempo de ejecución. Los genéricos son invariantes para evitar estos problemas.

Covariante: acepta subtipos. Contravariante: acepta supertipos. Invariante: no acepta ninguno.

13. Java permite recuperar covarianza y contravarianza en tipos genéricos de forma controlada mediante **wildcards**. ¿Qué es un wildcard (?)? Muestra la diferencia entre **List<? extends T>** y **List<? super T>**,

indicando en qué casos se usa cada uno. Pon dos ejemplos: (i) un método que reciba una lista de números y calcule su suma, usando `? extends`; (ii) un método que reciba una lista y le añada varios números enteros, usando `? super`.

## Respuesta

Un wildcard `?` representa un tipo desconocido.

`? extends T` permite leer elementos como tipo `T`, pero no añadirlos. `? super T` permite añadir elementos de tipo `T`, pero no leerlos con precisión.

```
public static double suma(List<? extends Number> lista) {
 double total = 0;
 for (Number n : lista) total += n.doubleValue();
 return total;
}
public static void addNumeros(List<? super Integer> lista) {
 lista.add(1);
 lista.add(2);
}
```

Esto permite controlar la seguridad de tipos de forma flexible.